Adaptive Few-Shot Prompting with Ensemble Retrieval for Financial Numerical Reasoning

Anonymous ACL submission

Abstract

Since their introduction, large language models (LLMs) have consistently improved in their capabilities. However, numerical reasoning remains a challenging area. Techniques like chain of thought prompting, tree of thought prompting, and program of thought prompting guide LLMs through intermediate reasoning steps. While in-context learning with few-shot prompting has improved performance, LLMs still lag behind state-of-the-art models on datasets such as FinQA and ConvFinQA. In this paper, we introduce a novel two-step framework to enhance LLM's capabilities in financial numerical reasoning. The first step utilizes a novel ensemble-based retriever to extract relevant facts from unstructured data, including both text and tables. This is followed by context-aware Program of Thought prompting with dynamic selection of in-context examples using a policy gradient method. Our model achieves a new state-of-the-art performance on both FinQA and ConvFinQA, surpassing previous benchmarks with execution accuracy improvements of 6.47% and 4.30%, respectively.

1 Introduction

Numerical reasoning has long been a challenging task in artificial intelligence. It is more complex than traditional question answering (Rajpurkar et al.; Yang et al.) because it involves identifying relevant information from diverse sources, such as tables and unstructured texts, and then constructing a coherent reasoning path to integrate all the necessary information. The finance domain inherently requires intricate numerical reasoning, such as the quantitative analysis of financial reports. Recently, datasets like FinQA (Chen et al., 2021b) and ConvFinQA (Chen et al., 2022) have been developed to benchmark the abilities of deep learning models in performing numerical and arithmetic reasoning on financial documents.

A common approach to tackle this task is the retriever-generator question-answering framework, initially introduced by Chen et al. (2021b). This framework operates in two stages: first, a retriever is trained to extract relevant information from documents; second, a generator is trained to create executable programs using the retrieved information. Implementations based on pre-trained masked language models and ensemble techniques (Wang et al.; Zhang et al.; Wang et al.) have significantly improved accuracy in this task. Li et al. (2022) proposed an enhanced framework that incorporates a dynamic retriever reranker generator, which refines each generation step by dynamically reranking the retrieved information. The current state-of-the-art model in these datasets is the APOLLO model (Sun et al., 2024b), which employs a number-aware retriever and a program-augmented generator, thereby boosting overall execution accuracy.

In recent years, large language models (LLMs) have achieved remarkable advancements in natural language processing (NLP). These models have shown significant progress in tasks such as language translation, content generation, and contextual comprehension. However, despite their impressive capabilities in various NLP tasks, numerical reasoning has often been considered a weak area for LLMs (Huang and Chang, 2022).

Reasoning tasks typically involve multiple steps, including gathering information, formulating arguments, evaluating them, and using logic to arrive at a conclusion or answer. Unlike other NLP tasks, numerical reasoning problems have only one correct answer, making them particularly challenging. A significant obstacle in numerical reasoning is its sensitivity to individual errors; even a minor mistake in sequence generation can lead to an entirely incorrect solution. This sensitivity complicates the generation of accurate solutions by LLMs. Interestingly, reasoning is observed to be one of the

emergent abilities in large-scale models, particularly those with over 100 billion parameters (Wei et al., 2022a).

To address these challenges, LLMs have been fine-tuned to produce explicit reasoning steps through few-shot prompting. Specifically, when provided with a few examples containing inputs, natural language rationales, and outputs, LLMs can mimic these demonstrations to generate both rationales and answers. This prompting method, later extended as Chain-of-Thought (CoT) (Wei et al., 2022b), achieves state-of-the-art performance across a broad range of textual and numerical reasoning datasets. Building on this approach, the Program of Thought (Chen et al., 2023) fine-tunes LLMs to generate executable code and utilizes an external Python interpreter to compute the final answer. This helps mitigate minor errors or numerical mistakes to which LLMs are prone, thereby improving accuracy.

Existing methods based on LLM prompting have achieved significantly better results compared to earlier LLM-based approaches but still fall short of the state-of-the-art model APOLLO (Sun et al., 2024b) on financial numerical reasoning datasets. Current prompting methods provide complete textual and tabular data within the prompt, which proves to be ineffective. As highlighted in the Program of Thought (POT) (Chen et al., 2023) work, the model often makes grounding errors, meaning it frequently fails to correctly extract the relevant numerical values from the provided data. This issue motivated us to develop a two-step framework featuring a novel ensemble-based retriever module, followed by Program of Thought prompting with contextually aware in-context examples to achieve state-of-the-art accuracy. We enhance prompting by implementing a dynamic in-context example selection policy, leveraging a diverse training samples collected through clustering techniques.

As outlined in existing works, scoring-based retrievers (Chen et al., Sun et al.) use a pre-trained language model along with a single layer of neural network to assign scores to each fact. In contrast, our proposed retriever leverages prompting and instruction-based fine-tuning of LLMs for relevant fact extraction. This approach allows for more precise and contextually relevant fact extraction from unstructured data, as opposed to the fixed-line selection process inherent in scoring-based methods. The current state-of-the-art APOLLO model employs a number-aware strategy to enhance the robustness of fact extraction. While effective, our approach does not rely on assumptions about the nature of the facts being extracted, making it more applicable to a wider range.

Our contributions can be summarized as follows:

- We propose a novel ensemble-based retrieval method involving instruction tuning of LLMs to extract relevant facts from extensive textual and tabular data to answer specific questions.
- We employ enhanced dynamic prompting by selecting in-context examples using a gradient-based policy from a diverse set of training samples collected through clustering techniques.
- Our approach[1] achieves a new state-of-the-art with an execution accuracy of **75.67** (**6.47%** gain) on FinQA, and **82.15** (**4.30%** gain) on ConvFinQA dataset.

2 Related Works

Developing models for numerical reasoning has attracted the AI community since the last century. Several QA datasets like ComplexWebQuestions (Talmor and Berant, 2018), TabFact (Chen et al., 2019), WikiTableQuestions (Pasupat and Liang, 2015) have been created to address numerical understanding and calculations. Numerical reasoning in math word problems has been a challenging task that has been addressed in (Huang et al., 2024), (Sun et al., 2024a), (Liu et al., 2020) for many years and there exist several benchmark datasets including MathQA (Amini et al., 2019) and MaWPS (Koncel-Kedziorski et al., 2016). (Lu et al., 2023) present Tabular Math Word Problems (TABMWP) dataset that require mathematical reasoning on both textual and tabular data. Long-form numerical reasoning is more challenging than MWP. FinQA (Chen et al., 2021b) and ConvFinQA (Chen et al., 2022), which are complex question-answering and conversational numerical reasoning tasks for financial reports fall under long-form numerical reasoning task. Several approaches ((Zhang et al., 2022), (Wang et al., 2022a), (Zhang and Moshfeghi, 2022), (Sun et al., 2024b)) are developed for these financial reasoning tasks. (Wei et al., 2022b) introduced chain-of-thought strategy to improve reasoning ability. (Chen et al., 2023) introduced better alternative to chain-of-thought prompting known as program of thought prompting.

[1]The data and code are available anonymously at https://anonymous.4open.science/r/Financial_Numerical_Reasoning-70B3/README.md

3 Methodology

Our proposed framework for financial numerical reasoning task is divided into two phases, a supervised generative relevant fact retriever phase, and a target answer computation phase as depicted in Figure 1.

3.1 Task Formulation

In this task, the input consists of a structured table T containing rows and columns of data, textual information, and a natural language question Q. The objective is to generate an output Y that correctly answers the question by leveraging the information encoded within the table and the textual content. Given the complexity of this task, we decompose it into two primary components:

Relevant Fact Retriever: This involves identifying the relevant facts from the input data.

Target Answer Computation: This involves developing and executing a program using the retrieved facts, in-context examples and the given question to derive the final answer.

3.2 Relevant Fact Retriever

We use fine-tuning based retrieval methods to extract the relevant facts for the question. To further enhance the retrieved facts, we propose a novel ensemble method.

3.2.1 FLAN-T5 and Mistral based Retrieval

We fine-tune FLAN-T5 Large (Chung et al., 2022) and Mistral-7B Instruct v0.2 (Jiang et al., 2023) models to retrieve relevant facts. Our choice of these large language models stems from their extensive pretraining through instruction tuning on a wide range of tasks. FLAN-T5 Large, instruction-tuned on over 1.8k tasks, demonstrates superior zero-shot and few-shot performance compared to its non-instruction-tuned counterpart, T5 Large (Raffel et al., 2023). Mistral-7B Instruct v0.2 is an instruction-finetuned version of Mistral-7B (Jiang et al., 2023), which outperforms models such as Llama-2-13B (Touvron et al., 2023b) across all tested benchmarks, and surpasses the LlaMa-1-34B (Touvron et al., 2023a) in mathematics and code generation. We refer FLAN-T5-Large and Mistral-7B Instruct v0.2 as FLAN-T5 and Mistral, respectively, in the remaining sections.

The Input prompt format is shown in left hand side of Figure 1 and it includes the Instruction, Task description, Context, and Question. We conduct the fine-tuning in a parameter-efficient manner using Low-Rank Adaptation (LoRA) (Hu et al., 2021).

Formally, the models are fine-tuned via prompt $\mathbf{P}$ which contains instructions of the task, the table in textual format $\mathbf{TabText}$ and textual data $\mathbf{D}$. This can be represented as:

$$\mathbf{P} = F_p(\text{TabText} + \text{D})$$

where F_p denotes a function that generates a set instruction template for the "Input prompt" as shown in Figure 1 and '+' denotes the concatenation operation. During training, the model is trained to produce the ground truth relevant fact $\mathbf{GTFact}$ and during inference, the output on test samples is the retrieved fact $\mathbf{RetFact}$.

3.2.2 Ensemble

We observe differences between the facts retrieved by the FLAN-T5 and Mistral models, which becomes evident through various metrics such as cosine similarity with the ground truth relevant facts and numerical coverage (percentage of relevant numeric present in predicted facts) reported in Table 2. Please refer to 'Analysis of Retriever Modules' in Section 7 for more details. Mistral outperforms FLAN-T5 in numerical coverage but falls short in cosine similarity metric. FLAN-T5 predicted facts are closer to the ground truth relevant facts.

To address these issues, we propose an ensemble model that leverages the strengths of both Mistral and FLAN-T5. This model integrates their outputs into a single, coherent sequence of facts. We utilize a pretrained BART-large model (Lewis et al., 2020), which is further fine-tuned using an input template (Ensemble Prompt), as illustrated in Figure 1.

Formally, let us denote the retrieved facts from FLAN-T5 as $\mathbf{RetFactF}$ and that from Mistral as $\mathbf{RetFactM}$. During the training, the BART-large model is trained to produce ground truth relevant fact $\mathbf{GTFact}$ given an input sequence $\mathbf{X}$ where:

$$\mathbf{X} = F_p\ (\text{Q} + \text{RetFactF} + \text{RetFactM})$$

where F_p denotes a function that generates a set instruction for the "Ensemble Prompt" as shown in Figure 1 and '+' denotes concatenation operation.

During the inference phase, the model produces the ensembled facts $\mathbf{RetFactE}$ as the output for the test samples. To finalize the retrieved facts, we proceed as follows:

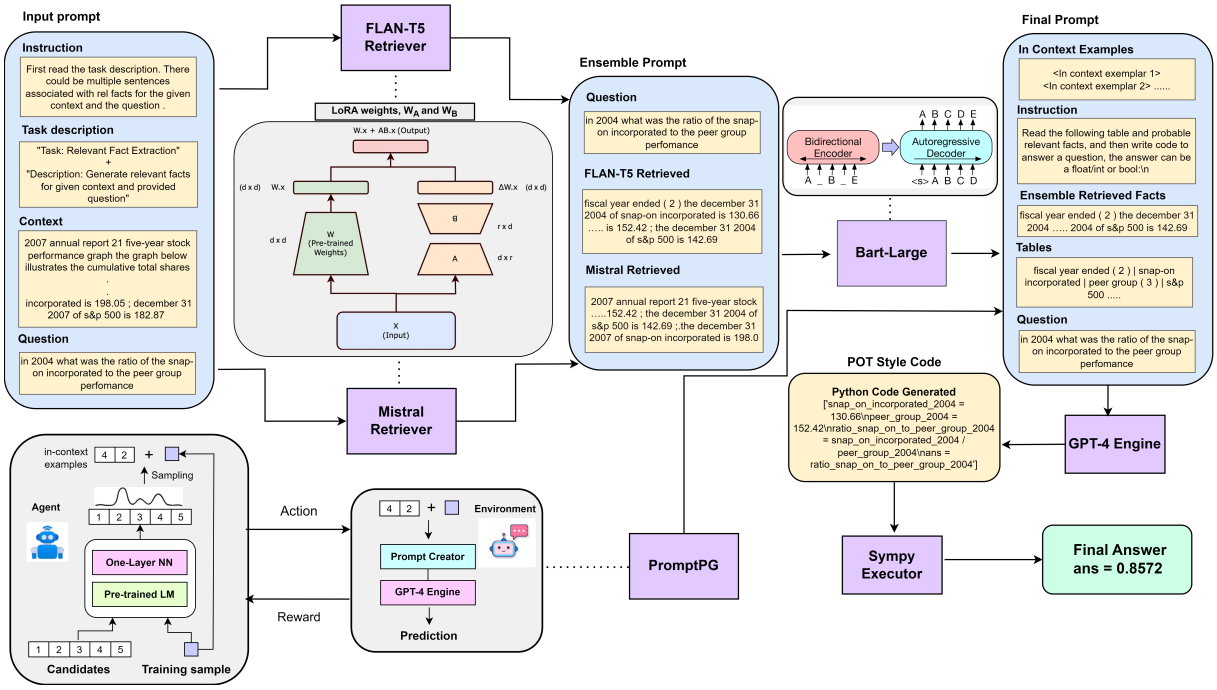


Figure 1: Architecture diagram of our proposed framework. FLAN-T5 and Mistral process task-specific instructions, context (text and tables), and questions to generate relevant facts. The Ensemble module refines these facts. GPT-4 generates POT style code using a Final Prompt with in-context examples from trained PromptPG and ensembled facts. The SymPy executor then interprets this code to produce the final answer.

First, we generate embeddings for each sentence in the context containing both tabular and textual data, in **RetFactE** using Sentence BERT (Reimers and Gurevych, 2019).

Next, we match each sentence in **RetFactE** with the sentence from the Context that has the highest cosine similarity. We denote this set of matched sentences as $\mathbf{RetFactMatched} = \{r_1, r_2, r_3, \ldots\}$, where the sequence $\{r_1, r_2, r_3, \ldots\}$ represents the matched sentences from the context. This step mitigates the issue of incomplete outputs from trained models and provide more complete and accurate retrieved facts.

3.3 Target Answer Computation

The initial step in this stage involves the dynamic selection of in-context examples. We use Program of Thought (POT) style prompting to generate Python code by GPT-4 engine (OpenAI, 2024), which is then executed by an external Python interpreter to obtain the final results. Unlike the static prompts used in POT, we leverage dynamic prompting to enhance the accuracy.

3.3.1 Dynamic In-Context Example Selection

We retrained the PromptPG framework (Lu et al., 2023) to learn the optimal policy for our dataset. This trained policy dynamically provides in-context examples tailored for each test sample. To create these training examples, we use agglomerative clustering (Müllner, 2011) on questions represented by Sentence BERT embeddings (Reimers and Gurevych, 2019), ensuring a diverse representation of question types. We identify 50 clusters and select a representative question for each cluster by choosing the one nearest to the centroid. Please refer Appendix A.1 to visualize some clusters.

Formally, given a problem p_i, we want the agent to find K in-context examples $e_i = \{e_i^1, e_i^2, ..., e_i^K\}$ using policy gradient techniques from a candidate pool E_{cand}, and generate the answer $\hat{a}_i$, maximizing a reward $r_i = R(\hat{a}_i|p_i)$ with the help of GPT-4 engine. The in-context examples are selected according to a policy

$$e_i^k \sim \pi_\theta(e_i|p_i), \quad e_i^k \in E_{\text{cand}}, \\ e_i^k \text{ are independent for } k = \{1, 2, ..., K\} \tag{1}$$

where θ are the policy's parameters. The answer is generated through: $\hat{a}_i = \text{GPT-4}(e_i, p_i)$ using the selected examples and the given problem as

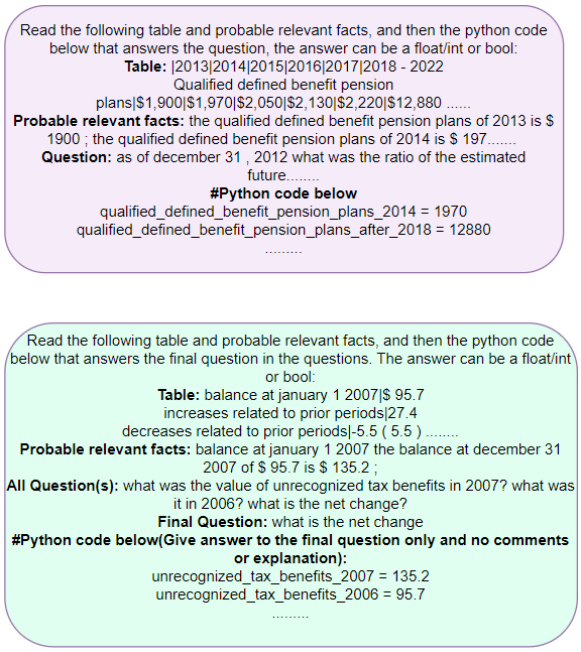


Figure 2: Exemplar in Program of Thought Prompt style, first one is for FinQA, second one is for ConvFinQA

the input prompt. The reward is then computed by evaluating the generated answer $\hat{a}_i$ with respect to the ground truth answer a_i:

$$r_i = R(\hat{a}_i|p_i) = \text{SCORE}(\hat{a}_i, a_i), \quad r_i \in \{-1, 1\}.$$

The function SCORE() returns a reward of 1 if the generated answer aligned with the label and -1 otherwise. During training, the goal is to maximize the expected reward of the generated answer under the policy $E_{e_i \sim \pi_\theta(e_i|p_i)}[R(\text{GPT-4}(e_i, p_i))]$.

3.3.2 Program of Thought based Prompting

We employ Program of Thought (POT) prompting (Chen et al., 2023) to generate code for the questions. In POT, language models express reasoning steps as Python programs as it often yields better results because the LLM focuses solely on generating the reasoning process through code. In the final prompt (broadly demonstrated as "**Final Prompt**" in Figure 1), we incorporate dynamically retrieved in-context examples in the format shown in Figure 2. We include the information regarding the question to be answered in the same format as in Figure 2 but without the final python code and we pass it to the GPT-4 engine. As shown in Figure 2, we use a slightly different prompt for ConvFinQA exemplars as it contains conversational questions.

Python Interpreter: Program-of-Thoughts (PoT) prompting delegates computation steps to an external language interpreter. SymPy-based code executor, is used to decode the reasoning steps and achieve final answer. SymPy (Meurer et al., 2017) is an open-source Python library for symbolic mathematics, enabling precise manipulation and computation of mathematical expressions.

4 Dataset & Evaluation Metrics

We perform our experiments on two datasets: FinQA (Chen et al., 2021b) and ConvFinQA (Chen et al., 2022).

FinQA: FinQA is a dataset of numerical reasoning over long-form financial data, containing 8,281 financial reports, along with their QA pairs annotated by financial experts. The data is divided into training, development, and test sets, containing 6,251, 883, and 1,147 instances respectively, following a 75%/10 %/15% split.

ConvFinQA: Conversational Finance Question Answering (ConvFinQA) is a dataset of conversational long-form numerical reasoning over financial data with 2,715 simple and 1,177 hybrid conversations, consisting 14,115 questions, divided into 3,037/421/434 for train/dev/test sets. For this dataset, we report results on dev set as the test set does not contain the executed answer and we can only attain accuracy on test set by submitting the generated program in a specific format and not in python code format we utilize.

Evaluation Metrics: We adopt execution accuracy as our evaluation metrics following the previous works of (Sun et al., 2024b), (Chen et al., 2023), (Lu et al., 2023) on this task.

5 Baselines

We compare the performance of our proposed model against several competitive models.

APOLLO (Sun et al., 2024b): First, we compare our results with the recent state-of-the-art model on two datasets. The APOLLO model incorporates a number-aware negative sampling strategy for the retriever and employs consistency-based reinforcement learning for the generator. **BloombergGPT** (Wu et al., 2023): A large language model specifically designed for financial tasks. **GPT-3.5-turbo** (Ouyang et al., 2022): A 175-billion parameter LLM trained on diverse textual data, evaluated in a zero-shot setting. **GPT-4** (OpenAI, 2024): A large-scale multimodal model evaluated in a few-shot setting. **Program-of-Thought** (Chen et al., 2023): Utilizes Codex to generate text and programming statements, ultimately providing an answer. The

results for **GPT-3.5-turbo** and **GPT-4** are directly sourced from (Li et al., 2023). From Program-of-Thought, we also report results for **Codex** (Chen et al., 2021a), **GPT-3** (Brown et al., 2020), **PoT-Codex** (Chen et al., 2023), **PoT-SC-Codex** (Chen et al., 2023) with direct answer and chain of thought (CoT) (Wei et al., 2022b) reasoning strategy.

Among fine-tuning based methods, we compare the performance between below mentioned models. **FinQANet** (Chen et al., 2021b), **NeRd** (Ran et al., 2019), **Longformer** (Beltagy et al., 2020), **GPT-2** (Radford et al., 2019), **T5** (Raffel et al., 2020), **CellRetriever+UniLM** (Zhang and Moshfeghi, 2022), **ELASTIC** (Wang et al., 2022a), **TabT5** (Andrejczuk et al., 2022), **DyRRen** (Jie et al., 2022).

Human Performance: Includes both experts and non-experts performance for the FinQA dataset.

6 Experimental Setup

We performed all the experiments on Tesla V100 32G GPU obtaining the pre-trained checkpoints from the *Huggingface* Library[2] for FLAN-T5-Large (780M parameters) and Mistral-7B Instruct v0.2 (7B parameters). FLAN-T5-Large were instruction-tuned with LoRA paradigm (parameters reduced to $\approx$ 0.59M, training time 15 minutes/epoch) for 10 epochs with batch-size of 8, lr of $5e-4$ and rank set to 2. Mistral-7B Instruct v0.2 model (7 Billion parameters) was also instruction-tuned in 4-bit precision with LoRA paradigm (parameters reduced to $\approx$ 3.4M, training time 9 hours/epoch) for 5 epochs, with a batch size of 4, lr of $5e-4$, and rank as 2. For ensemble, the BART-large model (406M parameters, training time 7 minutes/epoch) was trained for 40 epochs with a batch size of 8, lr of $1e-5$. We set the length of output to be generated to maximum 150 tokens.

7 Results and Analysis

We report the results of our proposed model in Table 1 for both FinQA and ConvFinQA dataset. We observe that our approach achieves **6.47%** gain for FinQA and **4.30%** gain for ConvFinQA over the current best model APOLLO (Sun et al., 2024b). When compared with the best baseline under the Program of Thought, PoT-SC-Codex, our model demonstrates a significant improvement across both datasets. Codex variants generally performed lower, with Codex Direct achieving the lowest scores on both the datasets. GPT-4 with fixed few-shot prompting, exhibited competitive results. In comparison, GPT-3.5 turbo-instruct showed a better performance than GPT-3.5-turbo on FinQA. Among fine-tuning based methods, TabT5 performs best on FinQA. We believe that providing the questions in a conversational format actually helps the LLM to understand the step by step procedure to solve the problem which in turn leads to higher execution accuracy for ConvFinQA data.

Model	Dataset	
	FinQA	ConvFinQA
	Exe Accuracy	Exe Accuracy
BloombergGPT	-	43.41
GPT-3.5-turbo	48.56	59.86
GPT-3.5-turbo-instruct	62.07	-
GPT-2	-	59.12
GPT-4	68.79	76.48
Program-of-Thought	68.10	67.30
T5	-	58.38
Retriever+NeRd	48.57	-
Longformer	21.90	-
FinQANet	61.24	68.32
ELASTIC	62.16	-
DyRRen	63.30	-
TabT5	70.79	-
CellRetriever+UniLM	68.00	-
Codex Direct	25.6	40.0
Codex CoT	40.4	45.6
GPT-3 Direct	14.4	29.1
GPT-3 CoT	26.1	37.4
PoT-Codex	64.5	64.6
Codex CoT-SC	44.4	47.9
PoT-SC-Codex	68.1	67.3
APOLLO	<u>71.07</u>	<u>78.76</u>
Ours	**75.67**	**82.15**
General Crowd Performance	50.68	-
Human Expert Performance	91.16	-

Table 1: Performance evaluation based on execution accuracy for FinQA dataset and ConvFinQA dataset. The best performance is highlighted in bold, and the strongest baseline result is underlined. The baseline results are reproduced from APOLLO (Sun et al., 2024b) and PoT (Chen et al., 2023) work.

Analysis of Retriever Modules: In FinQA and ConvFinQA datasets, the test samples contain the expected program steps, which denotes the numerical computation steps that would be required to arrive at the final answer. For numerical coverage, we assess whether all the numbers used in program steps are present in the retrieved facts. For each test sample, we find percentage of numbers in given program steps which are actually present in the predicted facts and average this across all test samples. This helps consolidate that our retrieved facts actu-

[2] https://huggingface.co/

ally contain all the required numerical data to solve the question. From Table 2, we observe that Mistral excels in numerical coverage compared to FLAN-T5 but performs worse in terms of cosine similarity. We believe this is because Mistral tends to generate additional lines and uses different wording from the context sentences, even if the semantic meaning remains the same. On the other hand, FLAN-T5 produces results that are closer to the ground truth relevant facts but sometimes outputs incomplete information. This observation is supported by the fact that average number of words present in Mistral predictions are higher than other models as shown in Table 2. The ensemble model (with lowest average words) achieves a higher cosine similarity compared to all individual retriever models, indicating that it produces facts closer to the ground truth. Although the recent state-of-the-art APOLLO retriever shows higher numerical coverage (attributed to its predictions containing more numbers and facts), but lower cosine similarity with ground truth facts compared to the ensemble retriever indicates that it provides a lot more irrelevant information. This is further evident by the average word count (highest for APOLLO) in the predictions, as shown in Table 2.

Retriever	Cosine Similarity		Numerical Coverage		Average Words	
	FinQA	ConvFinQA	FinQA	ConvFinQA	FinQA	ConvFinQA
Mistral	0.81	0.85	0.92	0.89	105.43	103.21
FLAN-T5	0.90	0.88	0.87	0.88	87.30	75.01
Ensemble	**0.92**	**0.90**	0.89	0.90	83.61	73.20
APOLLO	0.83	0.86	**0.92**	**0.91**	107.08	105.23

Table 2: Analysis of retrievers

In contrast, our model yields more targeted results. This precision might occasionally result in slightly lower numerical coverage or loss of some information. However, this trade-off leads to higher overall accuracy (refer Table 3), as our generator is less likely to be confused and can focus on relevant facts for most test samples.

8 Ablation Study

We now try out various ablations over our proposed model to understand the significance of different modules, effect of in-context example selection etc.

8.1 Retriever Modules Performance

We present the performance of individual retriever modules in Table 3. The comparison includes the final accuracy attained by APOLLO, Mistral, Flan, and the Ensemble-based retriever modules. As expected, the Ensemble-based retriever demonstrates the best performance. Interestingly, despite Mistral's higher numerical coverage compared to Flan, it achieves lower overall accuracy. We hypothesize that the excessive information provided by Mistral, beyond the relevant facts, may confuse the GPT-4 engine. It is important to note that all models surpass the current state of the art (APOLLO), highlighting the effectiveness of the proposed approach. The second-to-last row in Table 3 shows the result of the APOLLO retriever with our target answer computation module, which is lower than our proposed Ensemble retriever. However, the APOLLO retriever with our target computation module performs better than APOLLO's original methodology (first row of Table 3), indicating that our target computation approach is more accurate.

Retriever	Exe Accuracy	
	FinQA	ConvFinQA
APOLLO	71.07	78.76
Mistral	72.88	80.87
FLAN-T5	74.02	80.81
APOLLO with ours strategy	72.97	80.67
Ensemble	**75.67**	**82.15**

Table 3: Results for our ablation studies of model performance with different retriever modules

Question Types	Dataset	
	FinQA	ConvFinQA
	Exe Accuracy	Exe Accuracy
Table-only questions	81.02	88.19
Text-only questions	69.26	74.93
Table-Text questions	63.29	72.00
Numeric questions	75.24	82.17
Boolean questions	100.00	75.00
Program Steps		
1 step programs	80.58	84.17
2 step programs	72.61	78.79
>2 step programs	52.38	72.73
Programs with constants	54.76	76.75

Table 4: Performance comparison on various Question Types and Program Steps

8.2 Question Categorization vs Performance

We conducted an analysis of the model's performance by categorizing the questions into five distinct types: table-only, text-only, a combination of table and text, boolean, and numeric questions. The results of this analysis are detailed in Table 4. We observe that the model performs best on the Table-only questions and there is a drop in performance for questions involving both tables and text. For

FinQA, our model correctly answers all 20 boolean questions. Subsequently, we performed an ablation study focused on program steps. Although our framework does not integrate program steps, we examined the model's performance across different program steps using the FinQA and ConvFinQA dataset, which provides ground truth data for these steps. The model performs best on single-step programs, while questions involving more than two steps and constants are more challenging.

8.3 Effect of In-context Examples Selection

We experiment with different selection strategy of in-context examples and report the performance in Table 5. First, we use static in-context examples. Then, we try vanilla PromptPG which chooses initial in-context examples randomly (referred as random selection) for training. Next, we experiment with hybrid selection where in a four-shot setting, we select two most similar examples and two examples from a trained PromptPG policy network. Unlike the static approach used in the Program of Thought paradigm, our dynamic selection inspired by (Lu et al., 2023) with strategic initial in-context example selection via clustering, demonstrates superior performance than static, random and hybrid selection for both the datasets. Our experiment with changing the order of examples demonstrates that placing the highest-scoring example last (which we do in our model), thereby positioning the best context closest to the question, enhances performance. This highlights the importance of example order as shown in our approach compared to the third row of Table 5. These findings emphasize that dynamic, well-ordered, and strategically chosen initial training in-context examples are crucial for optimizing reasoning performance.

Model	Exe Accuracy	
	FinQA	ConvFinQA
Ours	75.67	82.15
w/ static in-context examples	72.01	78.53
w/ reverse ordering of in-context examples	74.51	81.53
w/ random selection	72.89	80.27
w/ hybrid selection	73.32	81.01

Table 5: Results for our ablation studies of in-context example selection

8.4 Sensitivity to Exemplars

To evaluate the sensitivity of our proposed model to different exemplars, we conducted a comprehensive few-shot analysis. For k-shot learning, we sampled k = (2, 3, 4, 5, 6) exemplars from the trained policy network. As shown in Figure 3, for both FinQA and ConvFinQA, we observe a significant drop in performance with only 2 exemplars provided, followed by a relatively stable trend that peaks with 4 exemplars.

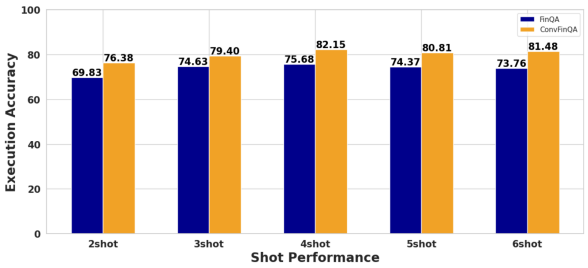


Figure 3: Few shot examplar analysis of FinQA and ConvFinQA dataset

9 Error Analysis

We conduct an error analysis of our predictions against the ground truth to identify various types of errors, as detailed in the Appendix A.4. Broadly, we categorize these errors into three types: fact retrieval errors, logical errors in the generated program, and issues with the ground truth or incomplete questions with majority being logical errors. Logical errors include grounding errors, where the retrieved fact is available, but the target computation module fails to recognize it as relevant for computing the final answer. Additionally, our analysis reveals that the model occasionally makes mistakes when faced with particularly complex questions or those requiring specialized financial knowledge. There are also instances where the ground truth itself is wrong or the given question is incomplete. Incorporating external financial knowledge could mitigate some of the errors.

10 Conclusion

In this work, we introduced a novel two-step framework to enhance LLMs ability to perform numerical reasoning in the financial domain. Our ensemble-based retriever, accurately extracts relevant information from unstructured data sources. Our dynamic in-context example selection, guided by a trained policy and clustering techniques, significantly improves the reasoning process. Future research will expand this framework to other domains needing complex numerical analysis and explore integrating external knowledge sources.

11 Limitations

In this work, we have not integrated external financial knowledge or human feedback in our framework. We evaluated our model on two financial reasoning datastes FinQA and ConvFinQA due to its complex nature. Though applicable, the model is not tested with other numerical reasoning datasets like GSM8K (Cobbe et al., 2021), AQuA (Ling et al., 2017), TaTQA (Zhu et al., 2021) etc. The interpretability of LLM reasoning processes and the potential propagation of errors within complex reasoning tasks requires further investigation.

References

Aida Amini, Saadia Gabriel, Shanchuan Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. MathQA: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367, Minneapolis, Minnesota. Association for Computational Linguistics.

Ewa Andrejczuk, Julian Eisenschlos, Francesco Piccinno, Syrine Krichene, and Yasemin Altun. 2022. Table-to-text generation and pre-training with TabT5. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 6758–6766, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021a. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks.

Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. 2019. Tabfact: A large-scale dataset for table-based fact verification. *arXiv preprint arXiv:1909.02164*.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021b. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6292, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2022. Scaling instruction-finetuned language models.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems, 2021. *URL https://arxiv.org/abs/2110.14168*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models.

Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403*.

Rikui Huang, Wei Wei, Xiaoye Qu, Wenfeng Xie, Xianling Mao, and Dangyang Chen. 2024. Joint multi-facts reasoning network for complex temporal question answering over knowledge graph. *arXiv preprint arXiv:2401.02212*.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b.

Zhanming Jie, Jierui Li, and Wei Lu. 2022. Learning to reason deductively: Math word problem solving as complex relation extraction. In *Proceedings of the*

60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 5944–5955, Dublin, Ireland. Association for Computational Linguistics.

Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. MAWPS: A math word problem repository. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1152–1157, San Diego, California. Association for Computational Linguistics.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880, Online. Association for Computational Linguistics.

Xianzhi Li, Samuel Chan, Xiaodan Zhu, Yulong Pei, Zhiqiang Ma, Xiaomo Liu, and Sameena Shah. 2023. Are ChatGPT and GPT-4 general-purpose solvers for financial text analytics? a study on several typical tasks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 408–422, Singapore. Association for Computational Linguistics.

Xiao Li, Yin Zhu, Sichen Liu, Jiangzhou Ju, Yuzhong Qu, and Gong Cheng. 2022. Dyrren: A dynamic retriever-reranker-generator model for numerical reasoning over tabular and textual data.

Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 158–167, Vancouver, Canada. Association for Computational Linguistics.

Daizong Liu, Xiaoye Qu, Jianfeng Dong, and Pan Zhou. 2020. Reasoning step-by-step: Temporal sentence localization in videos via deep rectification-modulation network. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 1841–1851, Barcelona, Spain (Online). International Committee on Computational Linguistics.

Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2023. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *International Conference on Learning Representations (ICLR)*.

Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, Ondřej Čertík, Sergey B. Kirpichev, Matthew Rocklin, AMiT Kumar, Sergiu Ivanov, Jason K. Moore, Sartaj Singh, Thilina Rathnayake, Sean Vig, Brian E. Granger, Richard P. Muller, Francesco Bonazzi, Harsh Gupta, Shivam Vats, Fredrik Johansson, Fabian Pedregosa, Matthew J. Curry, Andy R. Terrel, Štěpán Roučka, Ashutosh Saboo, Isuru Fernando, Sumith Kulal, Robert Cimrman, and Anthony Scopatz. 2017. Sympy: symbolic computing in python. *PeerJ Computer Science*, 3:e103.

Daniel Müllner. 2011. Modern hierarchical, agglomerative clustering algorithms. *arXiv preprint arXiv:1109.2378*.

OpenAI. 2024. Gpt-4 technical report.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.

Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1470–1480, Beijing, China. Association for Computational Linguistics.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the limits of transfer learning with a unified text-to-text transformer.

Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia. Association for Computational Linguistics.

Qiu Ran, Yankai Lin, Peng Li, Jie Zhou, and Zhiyuan Liu. 2019. NumNet: Machine reading comprehension with numerical reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International*

Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 2474–2484, Hong Kong, China. Association for Computational Linguistics.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks.

Jiashuo Sun, Yi Luo, Yeyun Gong, Chen Lin, Yelong Shen, Jian Guo, and Nan Duan. 2024a. Enhancing chain-of-thoughts prompting with iterative bootstrapping in large language models.

Jiashuo Sun, Hang Zhang, Chen Lin, Xiangdong Su, Yeyun Gong, and Jian Guo. 2024b. Apollo: An optimized training approach for long-form numerical reasoning.

Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 641–651, New Orleans, Louisiana. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023b. Llama 2: Open foundation and fine-tuned chat models.

Bin Wang, Jiangzhou Ju, Yunlin Mao, Xin-Yu Dai, Shujian Huang, and Jiajun Chen. 2022a. A numerical reasoning question answering system with fine-grained retriever and the ensemble of multiple generators for finqa.

Yanbo J. Wang, Yuming Li, Hui Qin, Yuhang Guan, and Sheng Chen. 2022b. A novel deberta-based model for financial question answering task.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022a. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022b. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.

Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Jiaxin Zhang and Yashar Moshfeghi. 2022. Elastic: Numerical reasoning with adaptive symbolic compiler. In *Advances in Neural Information Processing Systems*, volume 35, pages 12647–12661. Curran Associates, Inc.

Renhui Zhang, Youwei Zhang, and Yao Yu. 2022. A robustly optimized long text to math models for numerical reasoning on finqa.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, Online. Association for Computational Linguistics.

A Appendix

In this section, we provide some supplementary materials that enhance the content presented in the main paper.

A.1 Clustering of Questions

First, we embed the sentences using Sentence BERT embeddings. Next, we apply agglomerative clustering to the question embeddings, identifying 50 clusters. For each cluster, we select a representative question by choosing the one closest to the centroid. While we also tested with TF-IDF vectors, Sentence BERT yielded the best clusters with higher silhouette score. Figure 4 shows t-sne plot for visualization of some clusters.

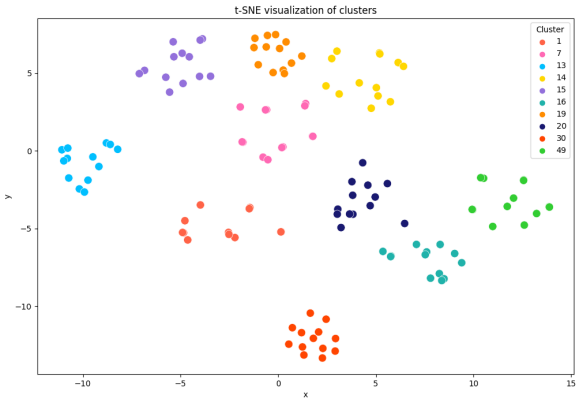


Figure 4: This image shows the Clusters formed of the embeddings of the train questions. We show only a few clusters to visualize.

A.2 Experimenting with other Ensemble Models

For ensemble though we reported only BART-large, we also experimented with BART-base and T5-large model. We achieve 73.67% accuracy with T5-large and 73.15% accuracy with BART-base on FinQA dataset which is lower than what we obtain using BART-large.

A.3 Experiments of POT with GPT-4

Though POT paper reports 74.0% execution accuracy on FinQA Dataset (outputs of which are missing in their codebase), when we ran with the given code we achieved only 69.38% execution accuracy. Even the current state of the art APOLLO paper does not report the POT + GPT-4 result. For this reason, we have not reported in our main results Table.

A.4 Error Analysis

In the following, we present few examples to showcase different errors exhibited by our model. Broadly, we categorize these errors into three types: wrong ground truth (Error Case-1), logical errors in the generated program (Error Case-2) and fact retrieval errors (Error Case-3). We highlight the retrieved relevant facts in color green. Figure 5, Figure 6 and Figure 7 represents Error Case-1. Figure 8, Figure 9, Figure 10, Figure 11 and Figure 12 showcase Error Case-2. Figure 13 shows an example of Error Case-3.

(In thousands)	Net Undeveloped Acres Expiring 2014	2015	2016
U.S.	145	60	46
E.G. [a]	36	—	—
Other Africa	189	2,605	189
Total Africa	225	2,605	189
Total Europe	216	372	1
Other International	—	20	—
Worldwide	586	3,057	236

[a] An exploratory well is planned on this acreage in 2014.

Question: what was average worldwide net acreage expiring in the total three year period , in millions?

Generated Program:

```
worldwide_acres_expiring_2014 = 586
worldwide_acres_expiring_2015 = 3057
worldwide_acres_expiring_2016 = 236
total_worldwide_acres_expiring = worldwide_acres_expiring_2014 + worldwide_acres_expiring_2015 + worldwide_acres_expiring_2016
average_worldwide_acres_expiring = total_worldwide_acres_expiring / 3
#Convert to millions
average_worldwide_acres_expiring_in_millions = average_worldwide_acres_expiring / 1000
ans = average_worldwide_acres_expiring_in_millions
```

Executed Answer: 1.293
Ground Truth Answer: 3879

Figure 5: Error Case-1: Wrong Ground Truth. The Ground truth answer doesn't convert units to millions and fails to divide by 3 to take average across years

Snap-on has paid consecutive quarterly cash dividends, without interruption or reduction, since 1939. Cash dividends paid in 2007, 2006 and 2005 totaled $64.8 million, $63.6 million and $57.8 million, respectively. On November 1, 2007, the company announced that its Board increased the quarterly cash dividend by 11.1% to $0.30 per share ($1.20 per share per year). At the beginning of fiscal 2006, the company's Board increased the quarterly cash dividend by 8% to $0.27 per share ($1.08 per share per year).

	2007	2006	2005
Cash dividends paid per common share	$ 1.11	$ 1.08	$ 1.00
Cash dividends paid as a percent of prior-year retained earnings	5.5%	5.6%	5.2%

Question: what was the percentage change in the cash dividends paid per common share from 2006 to 2007
Generated Program:

```
cash_dividends_per_share_2006 = 1.08
cash_dividends_per_share_2007 = 1.11
percentage_change_in_dividends = (cash_dividends_per_share_2007 - cash_dividends_per_share_2006) / cash_dividends_per_share_2006
ans = percentage_change_in_dividends
```

Executed Answer: 0.027777777777778
Ground Truth Answer: 2.02778

Figure 6: Error Case-1 (Ground truth wrong): Our generated answer correctly gives the change. The ground truth answer instead adds the values of 2006 and 2007 and divides by value of 2006 ((1.11 + 1.08)/1.08 = 2.02778)

Net revenue consists of operating revenues net of: 1) fuel, fuel-related expenses, and gas purchased for resale, 2) purchased power expenses, and 3) other regulatory credits. Following is an analysis of the change in net revenue comparing 2017 to 2016.

	Amount (In Millions)
2016 net revenue	$705.4
Volume/weather	(18.2)
Retail electric price	13.5
Other	2.4
2017 net revenue	**$703.1**

Question: what was the average net revenue between 2016 and 2017 in millions
Generated Program:

```
net_revenue_2016 = 705.4
net_revenue_2017 = 703.1
average_net_revenue = (net_revenue_2016 + net_revenue_2017) / 2
ans = average_net_revenue
```

Executed Answer: 704.25
Ground Truth Anwser: 705.25

Figure 7: Error Case-1 (Wrong Ground Truth): Though our retrieved relevant facts, generated program and executed answer is correct, there is mistake in the ground truth calculation of final answer.

Recourse Debt Transactions

During 2010, the Company redeemed $690 million aggregate principal of its 8.75% Second Priority Senior Secured Notes due 2013 ("the 2013 Notes"). The 2013 Notes were redeemed at a redemption price equal to 101.458% of the principal amount redeemed. The Company recognized a pre-tax loss on the redemption of the 2013 Notes of $15 million for the year ended December 31, 2010, which is included in "Other expense" in the accompanying Consolidated Statement of Operations.

On July 29, 2010, the Company entered into a second amendment ("Amendment No. 2") to the Fourth Amended and Restated Credit and Reimbursement Agreement, dated as of July 29, 2008, among the Company, various subsidiary guarantors and various lending institutions (the "Existing Credit Agreement") that amends and restates the Existing Credit Agreement (as so amended and restated by Amendment No. 2, the "Fifth Amended and Restated Credit Agreement"). The Fifth Amended and Restated Credit Agreement adjusted the terms and conditions of the Existing Credit Agreement, including the following changes:

Question: what is the annual interest cost savings by the company redeeming the 8.75% (8.75 %) second priority senior secured notes?
Generated Program:

```
principal_amount_redeemed = 690
annual_interest_rate = 8.75 / 100
annual_interest_cost_savings = principal_amount_redeemed *
annual_interest_rate
ans = annual_interest_cost_savings
```

Executed Answer: 60.3749999999999
Ground Truth Answer: 60375000

Figure 8: Error Case-2 (Logical error): Our generated program produces output in millions, but it should convert this output back to the base unit.

11. COMMITMENTS, CONTINGENCIES AND RISKS

OPERATING LEASES—As of December 31, 2002, the Company was obligated under long-term non-cancelable operating leases, primarily for office rental and site leases. Rental expense for operating leases, excluding amounts related to the sale/leaseback discussed below, was $31 million $32 million and $13 million in the years ended December 31, 2002, 2001and 2000, respectively, including commitments of businesses classified as discontinued amounting to $6 million in 2002, $16 million in 2001 and $6 million in 2000.

The future minimum lease commitments under these leases are as follows (in millions):

	Total	Discontinued Operations
2003	$ 30	$ 4
2004	20	4
2005	15	3
2006	11	1
2007	9	1
Thereafter	84	1
Total	$169	$14

SALE/LEASEBACK—In May 1999, a subsidiary of the Company acquired six electric generating stations from New York State Electric and Gas ("NYSEG"). Concurrently, the subsidiary sold two of the plants to an unrelated third party for $666 million and simultaneously entered into a leasing arrangement with the unrelated party. This transaction has been accounted for as a sale/leaseback with operating lease treatment. Rental expense was $54 million, $58 million and $54 million in 2002, 2001 and 2000, respectively.

Question: what was the decrease in rental expense (millions) for operating leases in continuing operations from 2003 to 2003?

Generated Program:

```
total_2003 = 30
discontinued_operations_2003 = 4
total_2002 = 31
discontinued_operations_2002 = 6
rental_expense_2003 = total_2003 - discontinued_operations_2003
rental_expense_2002 = total_2002 - discontinued_operations_2002
decrease_in_rental_expense = rental_expense_2002 - rental_expense_2003
ans = decrease_in_rental_expense
```

Executed Answer: -1

Ground Truth Answer: 1

Figure 9: Error Case-2(Logical Error): As the questions asks for amount of loss the answer should be reported in positive

A reconciliation of the statutory U.S. federal income tax rate to the Company's effective income tax rate is as follows:

	2017	2016	2015
Statutory U.S. rate	35.0 %	35.0 %	35.0 %
One time transition tax	9.1	-	-
State income taxes, net of federal benefit	0.4	0.9	0.4
Foreign operations	(7.4)	(8.0)	(8.1)
Domestic manufacturing deduction	(2.2)	(2.0)	(2.7)
R&D credit	(1.0)	(1.1)	(1.0)
Change in valuation allowance	0.2	(0.7)	(1.7)
Audit settlements and refunds	(0.1)	(0.2)	(0.7)
Excess stock benefits	(2.3)	-	-
Change in federal tax rate (deferred taxes)	(18.2)	-	-
Venezuela charges	-	-	4.5
Worthless stock deduction	-	0.4	(3.0)
Other, net	0.2	0.1	0.1
Effective income tax rate	13.7 %	24.4 %	22.8 %

Question: what is the difference between the statutory u.s . rate and the effective income tax rate in 2017?
Generated Program:

```
statutory_us_rate_2017=35.0
effective_income_tax_rate_2017=13.7
difference_statutory_and_effective_tax_rate_2017 = statutory_us_rate_2017 -
effective_income_tax_rate_2017
ans = difference_statutory_and_effective_tax_rate_2017
```

Executed Answer: 21.3
Ground truth Answer: 0.213

Figure 10: Error Case-2(Logical Error): Grounding error as the model doesnt understand that the rates are in percentage and hence needs extra division by 100 while reporting final answer

	2014	2015	2016	2017	2018	2019
JKHY	100.00	110.51	151.12	182.15	231.36	240.29
2019 Peer Group	100.00	126.23	142.94	166.15	224.73	281.09
2018 Peer Group	100.00	127.40	151.16	177.26	228.97	286.22
S&P 500	100.00	107.42	111.71	131.70	150.64	166.33

Question: what was the 5 year total return for jkhy?
Generated Program:

```
jkhy_2014 = 100.00
jkhy_2019 = 240.29
five_year_total_return_jkhy = (jkhy_2019 - jkhy_2014) / jkhy_2014
ans = five_year_total_return_jkhy
```

Executed Answer: 1.4029
Ground Truth Answer: 140.29

Figure 11: Error Case-2 (Logical Error): Formula used for total return is wrong

In millions	2010	2011	2012	2013	2014	Thereafter
Maturities of long-term debt (a)	$ 304	$ 574	$199	$131	$ 562	$ 7,263
Debt obligations with right of offset (b)	519	28	–	–	–	5,108
Lease obligations	177	148	124	96	79	184
Purchase obligations (c)	2,262	657	623	556	532	3,729
Total (d)	$3,262	$1,407	$946	$783	$1,173	$16,284

Question: considering the years 2011-2012 , what is the variation observed in the lease obligations?
Generated Program:

```
lease_obligations_2011 = 148
lease_obligations_2012 = 124
variation_in_lease_obligations = lease_obligations_2012 - lease_obligations_2011
ans = variation_in_lease_obligations
```

Executed program: -24
Ground Truth Program: 24

Figure 12: Error Case-2 (Logical Error): Difference can be reported by absolute number without sign

NOTE 7. RESTRUCTURING AND OTHER CHARGES (Continued)

Previously announced restructuring programs

The following table depicts the activity for previously announced restructuring programs through December 3, 1999:

	Accrued Balance at November 27 1998	Total Charges	Cash Payments	Adjustments	Accrued Balance at December 3 1999
Accrual related to previous restructurings . . .	$8,867	$—	$(6,221)	$(1,874)	$772

As of December 3, 1999, approximately $0.8 million in accrued restructuring costs remain related to the Company's fiscal 1998 restructuring program. This balance is comprised of $0.3 million in severance and related charges, $0.1 million in lease termination costs, and $0.4 million in canceled contracts. The majority of the accrual is expected to be paid by the first quarter of fiscal 2000. Cash payments for the twelve months ended December 3, 1999 related to the fiscal 1998 restructuring were $0.7 million, $3.6 million, and $0.4 million for severance and related charges, lease termination costs, and canceled

Question: what portion of the 1999 accrual balance related to restructurings is comprised of canceled contracts?
Generated Program:

```
# As there isn't any information provided about canceled contracts, it's impossible to calculate the requested portion. Assuming
none of the balance is from canceled contracts.
portion_of_1999_accrual_balance_related_to_canceled_contracts = 0
ans = portion_of_1999_accrual_balance_related_to_canceled_contracts
```

Executed Answer: 0
Ground Truth Answer: 0.5

Figure 13: Error Case-3 (No Retrieved facts match): No Retrieved Facts Extracted relevant to the question